



Challenge 1

ADVANCED ALGORITHMS AND PARALLEL PROGRAMMING

Fabrizio Ferrandi

Politecnico di Milano

Dipartimento di Elettronica, Informazione e Bioingegneria

fabrizio.ferrandi@polimi.it

- ❑ Groups up to 3 people
 - Changing team for the 2nd challenge is ok
- ❑ One week of time
- ❑ **3 points for free** in the 1st part of the exam
 - Skip selected questions
 - Valid any time in the whole academic year

- ❑ Material to be delivered on Webeep:
 - Copy of the Colab with code .pynb (no link)
 - Submitting a file other than .pynb is possible, but it requires prior discussion with me
 - Use comments
 - 24h grace period for configuration errors
 - Short PDF report (**max 2 pages**)
 - Experimental setup
 - Performance measurements
 - Explanation of design choices
 - No screenshots of the code!
- ❑ Deadline: **21st October(included)**

- ❑ Implement the Primality test described during the Primality testing lecture (see `lecture10PrimalityTest.pdf` pp. 12-14).
- ❑ The data type used for `n` can represent a number representable with 512 bits.
- ❑ Check the asymptotic complexity with Google Benchmarking library

```
primalityTest(int512 n) {  
    /* executes the randomized primality test for a chosen at  
    random*/  
  
    a = random(2, n-1);  
  
    isProbablyPrime = true;  
  
    result = power(a, n-1, n);  
  
    if (result != 1 || !isProbablyPrime)  
        return false;  
    else  
        return true;  
}
```

Resources

- Colab notebook on Divide and Conquer using Google Benchmark library: [link](#)
- Google Benchmark library documentation [link](#)
- Suggested library: Boost.Multiprecision [link](#)

```
#include <boost/multiprecision/cpp_int.hpp>
#include <iostream>
using namespace boost::multiprecision;
using namespace boost::multiprecision::literals;
using uint512 = number<cpp_int_backend<512, 512, unsigned_magnitude, unchecked, void>>;
using uint1024 = number<cpp_int_backend<1024, 1024, unsigned_magnitude, unchecked, void>>;

int main() {
    uint512 a = 1;
    a <= 511; // a = 2^511
    uint512 b = 0xAB54A98CEB1F0AD2_cppui128; // 128-bit constant promoted to 512-bit data type
    uint1024 mul_wide = uint1024(a) * uint1024(b);
    uint512 q = static_cast<uint512>(mul_wide / b);
    uint512 r = static_cast<uint512>(mul_wide % b);
    std::cout << "q == a ? " << (q == a) << "\n"; // 1 (true)
    std::cout << "r == 0 ? " << (r == 0) << "\n"; // 1 (true)
}
```

□ Goals:

- Implement the primality test algorithm discussed during the lecture
- Display the asymptotic complexity using the Google benchmark library
- Provide sufficient tests to validate the implementation